WORKSHOP ON MACHINE LEARNING FOR EARTH SYSTEM MODELLING · JÜLICH SUPERCOMPUTING CENTRE, FORSCHUNGSZENTRUM JÜLICH
UNIVERSITÄTSCLUB BONN · 24–26 AUGUST 2026

Agentic AI for Earth Science Software: a field report from building Worldscope

David Matthew Hall · NVIDIA · Berlin, Germany

An interactive Earth-system workbench in which AI agents are not chatbots bolted on, but first-class users of every capability.

See it run
Source & walkthrough video

WHAT IT DOES

Built in 4½ months. Every commit written by an AI agent.

MODELS ON A GLOBE

Puts NVIDIA's **Earth2Studio** toolset on an interactive 3D globe alongside satellite imagery, climate archives, live weather feeds, and observational data.

Forecast, diagnostic, and assimilation runs; ensemble members, vertical levels, and time are all selectable axes.

FORMATS & GRIDS

Reads, writes, and visualizes **NetCDF, GRIB2, GeoTIFF, and Zarr**, across many grid types — including **HEALPix rendered natively**, rather than interpolated to a lat–lon raster first.

Every data display carries units and a legend.

AGENTS AS USERS

Every capability is exposed through the **Model Context Protocol** — not a chat window added afterwards.

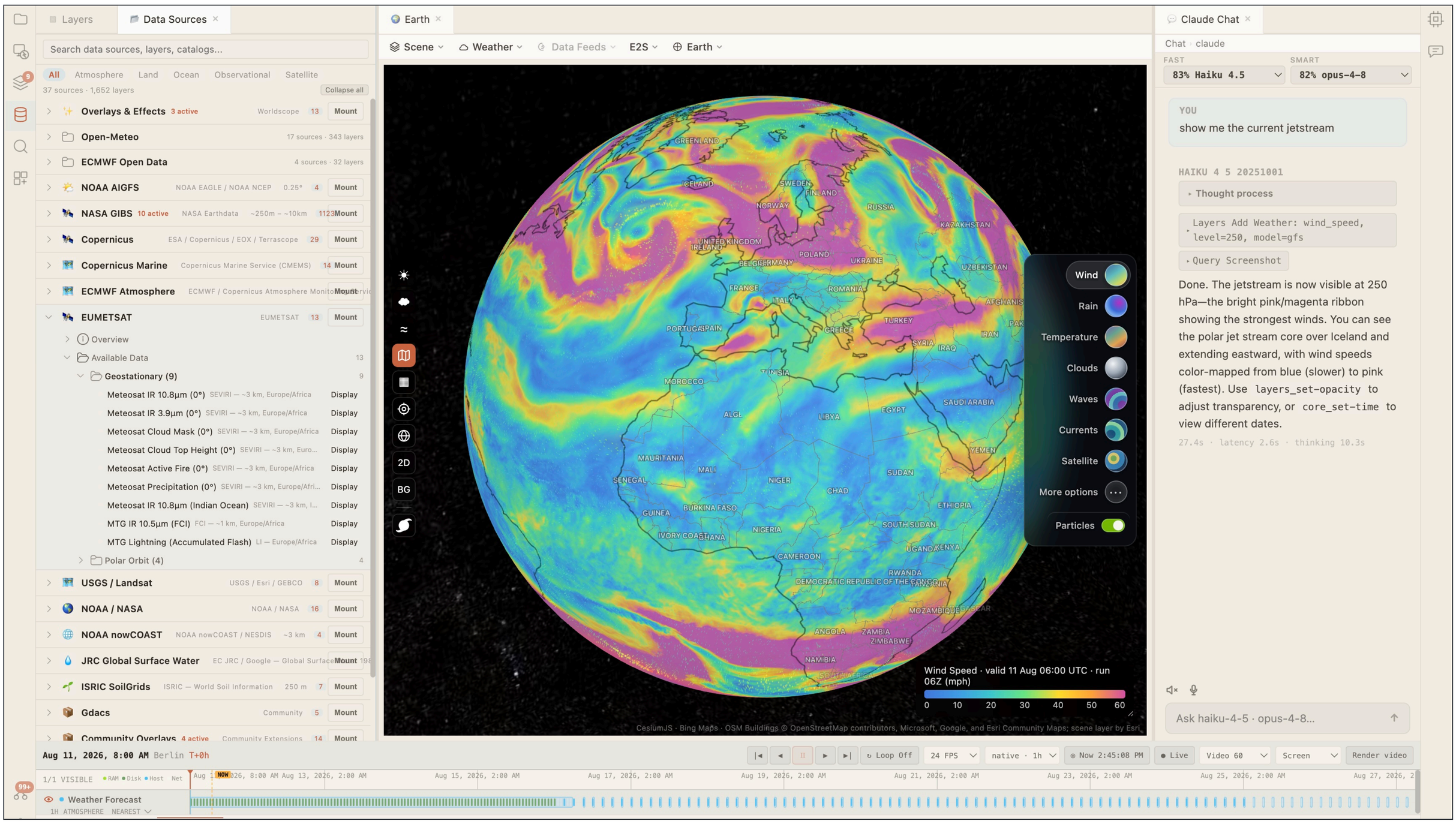
An agent can load a forecast, select a variable, run a model, compare results, and manipulate the same visualization a human sees. **Multiple agents can work one scene at once.**

RUNS WHERE YOU ARE

Lightweight enough for any device, while taking advantage of whatever NVIDIA hardware is present.

With a single desktop **DGX Spark** it runs the vast majority of AI weather models **locally**. Intended for open release.

4½ mo	1,444	181k	450	38	577	2 + 1
PROOF OF CONCEPT TO PLATFORM	COMMITS (100% AGENT-WRITTEN)	LINES OF SOURCE	AGENT-CALLABLE COMMANDS	COMMAND MODULES	TEST FILES	AI ASSISTANTS + ONE HUMAN



Worldscope. A global forecast field on the interactive globe, with the layer explorer at left and an agent session at right. The agent drives the same scene the human sees — same layers, same camera, same time cursor — through the Model Context Protocol, not a scripting sidecar.

One project is a data point. The pattern is the story.

I served as architect, tester, and director; the agents wrote the code. What follows is what that felt like from the inside — and the questions it raises for everyone here, because none of them are specific to this project.

1 The work did not vanish. It moved.

For well-specified features, agents produced implementations roughly **a hundred times faster** than I could have written them. That collapsed the cost of implementation and pushed the effort into specification, integration, and verification — none of which got cheaper.

2 Vision became the bottleneck.

Holding a coherent whole mattered **more** as implementation got cheaper, not less. Left to themselves, agents converge on the generic: the ordinary layout, the common grid, the default colormap, the solution that already exists somewhere in the training data.

3 Verification was the hardest part.

Especially for interactive behavior, and partly for a structural reason: the agents observed the browser through **still screenshots** and had to infer what happened in between. Features routinely looked correct and did not work; fixing one silently broke another.

4 Tests helped. Tests were not enough.

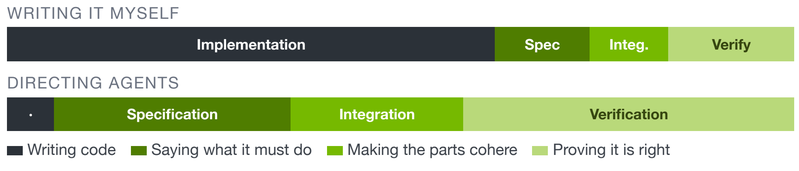
Agent-written tests caught regressions, but **could not establish that a complex interactive workflow behaved correctly end to end**. Plausible, runnable and quietly wrong is the failure mode that matters most in scientific software — and the one a green test suite does not catch.

5 Parallelism cuts both ways.

Running agents in parallel accelerated implementation while multiplying conflicting assumptions and integration regressions. Meanwhile the tools themselves were in constant flux, each new assistant arriving with its own strengths, weaknesses, and personality quirks.

Where the effort went

Schematic — impressions from one project, not measurements.



OPEN QUESTIONS FOR THIS COMMUNITY

Answers welcome. Arguments preferred.

I WHERE THE WORK GOES

→ What happens when AI agents are treated not as chatbots, but as *full participants* in an Earth-science workflow?

→ If writing code becomes a hundred times cheaper, which part of the scientific workflow becomes the bottleneck — and is any group staffed for it?

→ Is the durable artifact still the source code, or the *specification* that regenerates it?

→ What is scientific programming a skill *in*, once the programming is free?

II VERIFICATION & PHYSICALITY

→ How do you verify software you did not write, at the rate it is now written?

→ Who catches code that runs, passes its tests, and is *physically* wrong — flipped signs, unit slips, staggered-grid offsets, silent regridding, mishandled missing values, the wrong CRS?

→ Generation scales; checking does not. Is that asymmetry the real ceiling on agentic science?

→ What is the software equivalent of a physicality check — conservation, invariance, dimensional consistency, known-answer tests?

III EVALUATION & BENCHMARKS

→ We benchmark the models. Should we benchmark the *code that produces them* — and what would that benchmark measure?

→ Agents perceive interactive systems as still frames and infer the motion in between. What does that imply for evaluating anything time-dependent, interactive, or visual?

→ Do agent-built pipelines quietly alter our datasets — resampling, gap-filling, reprojecting — in ways our metrics never see?

→ If an agent writes the model, the training loop, *and* the evaluation, what independence is left?

IV PROVENANCE & REVIEW

→ If a published result rests on 180,000 lines no human has read line by line, what exactly was peer-reviewed?

→ What should journals and programme committees require — prompts, model versions, tool versions, transcripts, seeds?

→ Is agent-generated code reproducible once the agent that wrote it has been deprecated?

→ Who is the author of a scientific tool, and what counts as a citable software contribution now?

V THE GENERIC ATTRACTOR

→ Agents regress toward the mean of their training data. Does that quietly pull the field toward common grids, common formats, and mainstream methods?

→ How much of scientific novelty lives precisely in the choices an agent would talk you out of?

→ Does agentic development accelerate science, or accelerate its *consolidation*?

→ Do we get more science — or the same science, faster?

VI PEOPLE, ACCESS, COMMONS

→ If early-career scientists never write the code, where does the judgment to review it come from — and what do we train them to be good at?

→ Does this widen the gap between well-resourced and under-resourced groups, or erase it?

→ When a bespoke tool costs a weekend, do we still build shared community software — or does the toolchain quietly fragment?

→ Is scientific software still worth *maintaining*, or is it now disposable and regenerable?

As the cost of intelligence drops rapidly toward zero, the scarce resource turns out to be artistic vision and judgment.

If that is right —
are we selecting for it, teaching it, or rewarding it?

David Matthew Hall · NVIDIA · dhall@nvidia.com

Worldscope is intended for open release. Figures current as of the last commit before the meeting.